Reading Before Writing: Code-Comprehension Worksheet for Problem Set 2

Purpose

Problem Set 2 asks students to *write* vectorized R code. But writing presupposes being able to *read* — to look at a snippet, predict what kind of thing it returns, and explain what it's doing. This worksheet is meant to be done **away from a computer** — no running code, just reading and reasoning — so every dataset it refers to is shown here, printed out, the way it would look if you had typed it into the console yourself.

This worksheet selects six learning objectives from Pset 2 and, for each, gives five snippets of increasing sophistication.

How to use it: for each snippet, fill in:

- **Type of result** — e.g. a single number, a logical vector, a character string, a data frame, a factor, `NA`, an error, etc.
- **Explanation of task** — one or two sentences on what the code is doing and why, in your own words.

Only after you've written both should you check yourself by running the snippet.

Within each set of five, each snippet introduces exactly one new feature or idiom beyond the last one. All snippets use only base R and the **dslabs** `murders` dataset.

Objective A: Vectorized arithmetic with `seq` and `sum`

A.1

```
x <- 1:10
sum(x)
```

Type of result:

Explanation of task:

A.2

```
x <- seq(2, 20, by = 2)
sum(x)
```

Type of result:

Explanation of task:

A.3

```
n <- 50
sum(seq(1, n))
```

Type of result:

Explanation of task:

A.4

```
n <- 50
sum(1:n) == n * (n + 1) / 2
```

Type of result:

Explanation of task:

A.5

```
squares <- (1:10)^2
sum(squares) - sum(1:10)^2 / 10
```

Type of result:

Explanation of task:

Objective B: Logical subsetting/filtering of data frames

Before the snippets below, here is the dataset — this isn't one of the five to predict, just a reference to work from with the computer closed.

```
library(dslabs)
data(murders)
str(murders)
```

Output:

```
'data.frame':   51 obs. of  5 variables:
 $ state     : chr  "Alabama" "Alaska" "Arizona" "Arkansas" ...
 $ abb       : chr  "AL" "AK" "AZ" "AR" ...
 $ region    : Factor w/ 4 levels "Northeast","South",..: 2 4 4 2 4 4 1 2 2 2 ...
 $ population: num  4779736 710231 6392017 2915918 37253956 ...
 $ total     : num  135 19 232 93 1257 ...
```

```
head(murders)
```

Output:

```
       state abb region population total
1    Alabama  AL  South    4779736   135
2     Alaska  AK   West     710231    19
3    Arizona  AZ   West    6392017   232
4   Arkansas  AR  South    2915918    93
5 California  CA   West   37253956  1257
6   Colorado  CO   West    5029196    65
```

B.1

```
murders$population > 10000000
```

Type of result:

Explanation of task:

B.2

```
murders[murders$population > 10000000, ]
```

Type of result:

Explanation of task:

B.3

```
murders[murders$population > 10000000, c("state", "population")]
```

Type of result:

Explanation of task:

B.4

```
murders$rate <- murders$total / murders$population * 100000
murders[murders$rate < 1, "state"]
```

Type of result:

Explanation of task:

B.5

```
subset(murders, rate < 1 & region == "West")$state
```

Type of result:

Explanation of task:

Objective C: Membership testing with `%in%` and its negation

`state.abb` is a vector built into base R (part of the `datasets` package that loads automatically — no `library()` call needed) holding the 50 two-letter state abbreviations, in the same order as `state.name`.

```
state.abb
```

Output:

```
 [1] "AL" "AK" "AZ" "AR" "CA" "CO" "CT" "DE" "FL" "GA" "HI" "ID" "IL" "IN" "IA"
[16] "KS" "KY" "LA" "ME" "MD" "MA" "MI" "MN" "MS" "MO" "MT" "NE" "NV" "NH" "NJ"
[31] "NM" "NY" "NC" "ND" "OH" "OK" "OR" "PA" "RI" "SC" "SD" "TN" "TX" "UT" "VT"
[46] "VA" "WA" "WV" "WI" "WY"
```

C.1

```
c("MA", "ME", "ZZ") %in% state.abb
```

Type of result:

Explanation of task:

C.2

```
!(c("MA", "ME", "ZZ") %in% state.abb)
```

Type of result:

Explanation of task:

C.3

```
abbs <- c("MA", "ME", "ZZ")
abbs[!(abbs %in% state.abb)]
```

Type of result:

Explanation of task:

C.4

```
which(!(c("MA", "ME", "ZZ") %in% state.abb))
```

Type of result:

Explanation of task:

C.5

```
order <- c("California", "Texas", "New York")
murders[match(order, murders$state), "state"]
```

Type of result:

Explanation of task:

Objective D: Multi-category labeling with `ifelse`

D.1

```
x <- c(-2, 0, 3, 5)
ifelse(x > 0, "positive", "not positive")
```

Type of result:

Explanation of task:

D.2

```
rate <- c(0.5, 2, 6, 10)
ifelse(rate > 5, "high", ifelse(rate > 1, "medium", "low"))
```

Type of result:

Explanation of task:

D.3

A state's murder *rate* tells you how dangerous it is per capita, but the *number of people affected* also depends on how large the state is. Combine both:

```
national_rate <- mean(murders$rate)
elevated <- murders$rate > national_rate
populous <- murders$population > 5000000
ifelse(elevated & populous, "Large & Elevated", "Other")
```

Type of result:

Explanation of task:

D.4

```
burden <- ifelse(elevated & populous, "Large & Elevated",
  ifelse(elevated & !populous, "Small & Elevated", "Typical"))
table(burden)
```

Type of result:

Explanation of task:

D.5

```
cut(murders$rate, breaks = c(-Inf, 1, 3, 5, Inf),
    labels = c("Very Safe", "Safe", "Moderate", "High Risk"))
```

Type of result:

Explanation of task:

Objective E: Writing functions with conditionals and `NA` handling

E.1

```r
square <- function(x) x^2
square(4)
```

Type of result:

Explanation of task:

E.2

```r
classify <- function(x) {
  if (x > 0) "positive" else "not positive"
}
classify(-3)
```

Type of result:

Explanation of task:

E.3

```r
safe_sqrt <- function(x) {
  if (x < 0) return(NA)
  sqrt(x)
}
sapply(c(4, -1, 9), safe_sqrt)
```

Type of result:

Explanation of task:

E.4

```r
harmonic_mean <- function(x) {
  if (any(x <= 0)) return(NA)
  length(x) / sum(1 / x)
}
harmonic_mean(c(1, 2, 4, 8))
harmonic_mean(c(1, -2, 4))
```

Type of result:

Explanation of task:

E.5

```r
classify_state <- function(name) {
  idx <- which(murders$state == name)
  if (length(idx) == 0) return("State not found")
  r <- murders$rate[idx]
  if (r < 1) "Very Safe"
  else if (r < 3) "Safe"
  else if (r < 5) "Moderate"
  else "High Risk"
}
sapply(c("Vermont", "NotAState"), classify_state)
```

Type of result:

Explanation of task:

Objective F: Apply-family functions (`sapply`) combined with `table`

F.1

```r
sapply(1:5, function(x) x^2)
```

Type of result:

Explanation of task:

F.2

```r
sapply(murders$state, nchar)
```

Type of result:

Explanation of task:

F.3

```r
categories <- sapply(murders$rate, function(r) {
  if (r < 1) "Low"
  else if (r < 5) "Medium"
  else "High"
})
table(categories)
```

Type of result:

Explanation of task:

F.4

```
table(sapply(murders$rate, function(r) r < mean(murders$rate)))
```

Type of result:

Explanation of task:

F.5

```
sapply(split(murders$rate, murders$region), mean)
```

Type of result:

Explanation of task: